

Práctica 3 - Monitores

Sintaxis - estructura y comunicac

```
Monitor Nombre {
  variables permanentes del monitor
```

```
  Procedure uno {
    var locales
```

```
  //código de inicializac
  {
```

```
    Inicializac de variables
  }
```

```
}
```

Para tener en cuenta

No existen variables compartidas
Las variables permanentes del monitor solo se deben usar dentro del monitor

Si el monitor tiene un código de inicializac, hasta q' este no termine, no se atenderán llamados a los procedimientos

Los procesos interactúan entre ellos y con los RP compartidos a medio de los monitores haciendo llamados a estos

Sintaxis - sincronizac

```
Monitor Nombre {
  cond vc;
```

```
  Procedure uno() {
    wait(vc);
  }
```

```
  Procedure dos() {
    signal(vc);
    ...
    signal_all(vc);
  }
```

```
}
```

La EM es implícita dentro de un monitor al no poder ejecutar más de un llamado a un procedimiento a la vez, hasta q' no se termine el procedimiento o no se duerma en una variable condition, no se liberará

La sincronizac a condic es explícita a medio de variables conditions usadas. Son variables permanentes del monitor.

- wait(vc): duerme al proceso en la cola asociada a la variable condic (al final de la cola)

- signal(vc): despierta al primer proceso dormido en vc (al primero q' se había dormido) p/ q' completa nueva p/ acceder al monitor, y cuando lo haga, continúa con la instrucción después del wait

- signal_all(vc): despierta a todos los procesos dormidos en vc p/ q' todos pasen a competir a acceder nueva al monitor.

1) a) No funciona correctamente, en "while (cant > 0) wait (cola);", pero el segundo auto que entra podría quedar en loop por el while

b) Podría simplificarse a un único proceso PasarAlDiente().

c) La original del enunciado si, estaba de más xq' no se menciona el orden de llegada de la b) no lo repeta

if => p/ orden

while => NO requiere orden

- 2) a) proceso N procesos, monitor p/ la BD
b) Hazendo 5 consultas

proceso Proceso[i:=1..N]{

BBDD.entrar();
// hara consulta
BBDD.salir();
}

monitor BBDD{

cond consultas; int cant=5;

procedure entrar() {
if (cant==0) wait(cant);
cant--;

procedure salir() {
cant++;
signal(cant);
}

- 1) resuelve el a) como b) xq
no metodo de primera

- 3) b) Proceso Persona, monitor Fotopiedra, sacro = F/S.

proceso Persona[i:=1..N]{

Fotopiedra.usar();
Fotopiedra();
Fotopiedra liberar();
}

monitor Fotopiedra{

bool libre=true; cond file;

process usar() {
if (not libre) {
aprendo++;
wait(file);
}

else libre=false;

process liberar() {
if (aprendo > 0) {
aprendo--;
signal(cant);
} else libre=true;

Fotopiedra usar();
Fotopiedra();
Fotopiedra liberar();
es lo q genera el orden
de llegada.
Todos llegan y se encolan
Fotopiedra.fotopiedra();
una p. todos compiten
p/ entrar.

a) p

Monitor Fotopiedra{

Procedure fotopiedra();
esperaDoc();
}

c) Atended a persona de edad mayor.

```

process Persona [id = 1..N] {
  bool edad = ...;

  Fotocopiar (id, edad);
  Fotocopiar();
  Fotocopiar.liberar();
}

```

```

monitor Fotocopiar() {
  bool libre = true;  cond cola;
  int idAux, esperando = 0;  cola Ordenado (fila);

  procedure usar (idP, edad in int) {
    if (not libre) {
      inserta(fila, idP, edad);
      esperando ++;
      wait (cola);
    }
    else libre = false;
  }
}

```

```

procedure liberar() {
  if (esperando > 0) {
    esperando --;
    sacar(fila, idAux);
    signal (cola);
  }
  else libre = true;
}

```

d) Se debe respetar estricta el orden x id del proceso. (X no puede usar hasta q X-1 la haya usado)

```

process Persona [id = 1..N] {
  bool edad = ...;

  Fotocopiar (id);
  Fotocopiar();
  Fotocopiar.liberar();
}

```

```

monitor Fotocopiar() {
  int proximo = 1;
  cond espera[N];

  procedure usar (id in int) {
    if (id != proximo) {
      wait (espera[id]);
    }
  }

  procedure liberar() {
    proximo++;
    signal (espera[proximo]);
  }
}

```

e) Hay además un Empleado q le indica a cada persona cuando usar la fotocopadora.
 2 procesos $\Rightarrow$ Empleado + Persona Monitor $\Rightarrow$ Fotocopadora

process Persona $[i=1..N]$ {

Fotocopadora.usr();
 Fotocopar();
 Fotocopadora.librar();
 }

process Empleado() {

int i;
 for $(i=1..N)$ {
 Fotocopadora.asignar();
 }

monitor Fotocopadora {

int esperando=0; cond cliente, empleado, fin;

process usr() {
 signal(empleado);
 esperando++;
 wait(cliente);
 }

process librar() {
 signal(fin);
 }

process asignar() {
 if (esperando==0) {
 wait(empleado);
 }

esperando--;
 signal(cliente);
 wait(fin);
 }

f) Modificar e) p/ el caso de 10 empleados. El empleado le indica a la persona que fotocopadora usar y cuando.

process Persona $[i=1..N]$ {

Fotocopadora.usr();
 Fotocopar();
 Fotocopadora.dejar();
 }

process Empleado() {

int i;
 for $i=0..N-1$ {
 Fotocopadora.asignar();
 }

monitor Fotocopadora() {

int esperando=0;
 cond empleado, persona, fin;

procedure usr() {
 signal(empleado);
 esperando++;
 wait(persona);
 }

procedure dejar() {
 signal(fin);
 }

procedure asignar() {
 if (esperando==0) {
 wait(empleado);
 }
 esperando--;
 signal(persona);
 wait(fin);
 }

```

4) process Vehiculo [i := 1..N] {
    int peso = ...;
    Puesto ingreso (peso);
    delay (tiempo (cruce));
    Puesto use (peso);
}

```

```

Monitor Puesto {}
const max = 50000;
int pesoTotal = 0;
cond espera;
cole pesos;

```

```

procedure ingreso (peso: in int) {
    if (empty(pesos) AND (pesoTotal + peso < MAX)) {
        pesoTotal = pesoTotal + peso;
    } else {
        push(pesos, peso);
        wait (espera);
    }
}

```

```

procedure use (peso: in int) {
    int aux;
    pesoTotal = pesoTotal - peso;
    while (not empty(cole) AND (pesoTotal + primo(pesos) < MAX)) {
        pesoTotal = pesoTotal + primo(pesos);
        pop(pesos, aux);
        signal(pesos);
    }
}

```

5) a) único empleado

```

process Cliente [i := 1..N] {
    Mesa Atencion, ingreso();
    Entrega Papeles();
    Mesa Atencion, use();
}

```

```

process Empleado {
    int i;
    for (i := 1..N) {
        Mesa Atencion, atender();
    }
}

```

```

Monitor MesaAtencion {}
cond esperando, empleado;
int esperando = 0;

```

```

procedure ingreso() {
    signal(empleado);
    esperando ++;
    wait(persona);
}

```

```

procedure atender() {
    if (esperando == 0) {
        wait(empleado);
        esperando --;
        signal(persona);
    }
}

```

b) correlón c/ E empleados. NO deben terminar su ejecución

```

Proces Cliente [id: 1..N] {
  int idE;
  text papel, res;
  Correlón - Llegada (idE);
  Escritorio [idE]. atencion (papel, res);
}

```

```

Proces Empleado [id: 1..J] {
  text datos;

  while (true) {
    Correlón proximo (id);
    Escritorio [id]. esperar Datos (datos);
    res = resolver documentos;
    Escritorio [id]. enviar Resultado (res);
  }
}

```

```

Monitor Escritorio [id: 1..M] {
  cond vc Cliente, vc Empleado;
  text datos, resultados;
  boolean listo = false;
}

```

```

procedure atencion (D: in text, R: out text) {
  datos = D;
  listo = true;
  signal (vc Empleado);
  wait (vc Cliente);
  R = resultados;
  signal (vc Empleado);
}

```

```

procedure esperar Datos (D: out text) {
  if (not listo) wait (vc Empleado);
  D = datos;
}

```

```

procedure enviar Resultados (R: in text) {
  resultado = R;
  signal (vc Cliente);
  wait (vc Empleado);
  listo = false;
}

```

```

Monitor Correlón {
  cola elibres;
  cond esperaC;
  int esperando; cantLibres: 0;

  procedure Llegada (idE: out int) {
    if (cantLibres == 0) {
      esperando ++;
      wait (esperaC);
    } else {
      cantLibres --;
      pop (elibres, idE);
    }
  }
}

```

```

procedure proximo (idE: in out) {
  push (elibres, idE);
  if (esperando > 0) {
    esperando --;
    signal (esperaC);
  } else cantLibres ++;
}

```

c) Empleado se modifica while a for.

```

for [i: 1..N] {
  correlón proximo (id);
  Escritorio [id]. esperar Datos (datos);
  res = resolver documentos;
  Escritorio [id]. enviar Resultado (res);
}

```

consultar

HOJA N° 4

FECHA

```

6) proces Mueve [1..50]
    int nroGrupo;
    Mesa: obtenerNroGrupo(nroGrupo, i);
    //realiza tarea
    Mesa: entregarTarea(nroGrupo, i);
    proteje;
}

proces JTP
    Mesa: asignaGrupo();
    for [1..N]
        Mesa: entregaNota();
    }

Monitor Mesa { read operaJTP, operaCuerp;
    read operaGrupo; int operando=0;
    int puzos=25; alumnos[50]=0; read alumnos;
    int idReceptor; cola gruposEntregados;
    procedure obtenerGrupo(nro: out int; i: in int)
        operando++;
        if (operando < 50)
            wait (operaCuerp);
        else
            wait (operaJTP);
        read (operaGrupo);
        nro = nroGrupo[i];
    }
    procedure asignaGrupo()
        wait (operaJTP);
        for [1..50]
            signal (operaCuerp);
            signal (all);
    }

    procedure entregaTarea(nroGrupo: in int; id: in int; Holo: out int)
        alumnos[id] = nroGrupo;
        if (completaEntrega())
            signal (operaCuerp);
            gruposEntregados.push(idGrupo);
        wait (alumnos[id]);
        Holo = alumnos[id];
    }

    procedure entregaNota()
        int aux;
        if (empty(gruposEntregados)) wait (operaCuerp);
        aux = gruposEntregados.pop();
        LiberarMiembros(aux);
        proteje--;
    }

    if (operando < 50)
        wait (operaJTP);
    else {
        for [1..N]
            nroGrupos[i] = Asigna(NroGrupo);
        }
        signal (all (operaGrupo);
    }
}

```

vector JTP
asigna.

NOTA

```

e) process Persona[i:1..20]{
  at nro = 0;
  nro = DelEguipo();

  Coordinador := el Llegada(nro, cancha(nro));
  Cancha[cancha(nro)] = jugarPartido();
  delay(50');
}

```

```

Monitor Cancha[i:1..2]{
  at esperando = 0;
  cond Llegada;
  procedure jugarPartido();
  esperando++;
  if (esperando < 10){
    wait(Llegada);
  }
  else {
    signal-all(Llegada);
  }
}

```

```

Monitor Coordinador {
  at equipo[4] = {1,4,7,0};
  at canchaAsignada[4] = {1,4,7,1};
  cond jugadores[4];
  at equiposListos;
  at canchaLibre;

  procedure el Llegada(nro: in int, cancha: out int){
    at Lib;
    expect can Lib;
    if (equiposListos < 5){
      wait(jugadores[nro]);
    }
    else {
      push(equiposListos, nro);
      if (size(equiposListos) >= 2){
        pop(equiposListos, nro);
        pop(equiposListos, rival);
        canchaAsignada[nro] = canchaLibre;
        canchaAsignada[rival] = canchaLibre;
        canchaLibre = canchaLibre + 1;
        for (i=1..4) signal(jugadores[nro]);
        for (i=1..4) signal(jugadores[rival]);
      }
      else {
        wait(jugadores[nro]);
      }
      cancha = canchaAsignada[nro];
    }
  }
}

```

9) 3 procesos: Alumno, Profesor + monitor Exámenes

Proceso Alumno [i: 1..45] {

text examen;

Exámenes.obtenerExamen(examen);

//realizar examen

Exámenes.entregarExamen(examen, nota);

}

Proceso Profesor {

Exámenes.entregarExámenes();

}

Proceso Profesor {

for (i: 1..45) {

Exámenes.corrigeExamen();

}

Monitor Exámenes {

int esperando = 0; cola exámenes; cola notas;

cond alumnoEntregado: todos Disponibles;

cond examenEntregado: examen Corregido; cola entregados;

procedure obtenerExamen(examen: out text);

esperando ++;

if (esperando < 45) {

wait(alumnoEntregado);

} else {

signal(todosDisponibles);

} pop(entregados, examen);

procedure entregarExámenes();

wait(todosDisponibles);

for (i: 1..N) {

push(entregados);

signal(alumnoEntregado);

}

procedure entregarExamen(examen: in text; nota: out int);

entregados.push(examen);

signal(examenEntregado);

wait(examenCorregido);

pop(notas, nota);

}

procedure corrigeExamen();

text examen; int aux;

wait(examenEntregado);

pop(entregados, examen);

aux: corrigeExamen(examen);

push(notas, aux);

signal(examenCorregido);

}

10)

proceder Persona E := 1..N

int tiempoJuego := ...

Juego. Iniciar A Juego ();

// Juego. Juego

atray (tiempoJuego);

Juego. Iniciar B Juego ();

{

proceder Empleado

{for E := 1..N}

Juego. Iniciar Juego ();

{

(ver. 201)

mensaje al operador > 0?

collegio 03/10

Monitor Juego

int operador := 0;

cond. juegoLimpio, Mensaje Empleado

proceder Iniciar Juego ();

if (operador > 0) {

operador ++;

return; signal (Mensaje Empleado);

wait (juegoLimpio);

proceder Iniciar Juego ();

if (operador > 0) wait (Mensaje Empleado);

Declarar Juego ();

if (operador > 0) signal (juegoLimpio);

proceder usar De Juego ();

operador --;

signal (Mensaje Empleado);

{